Transport Layer

Anand Baswade

anand@iitbhilai.ac.in

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

TCP: Triggering congestion control

- Two ways to trigger a congestion notification in TCP – (1) RTO, (2) Duplicate ACK
- RTO: A sure indication of congestion, however time consuming
- Duplicate ACK: Receiver sends a duplicate ACK when it receives out of order segment
  - A loose way of indicating congestion
  - TCP arbitrarily assumes that THREE duplicate ACKs (DUPACKs) imply that a packet has been lost – triggers congestion control mechanism
  - The identity of the lost packet can be inferred – the very next packet in sequence
  - Retransmit the lost packet and trigger congestion control

TCP congestion control: AIMD

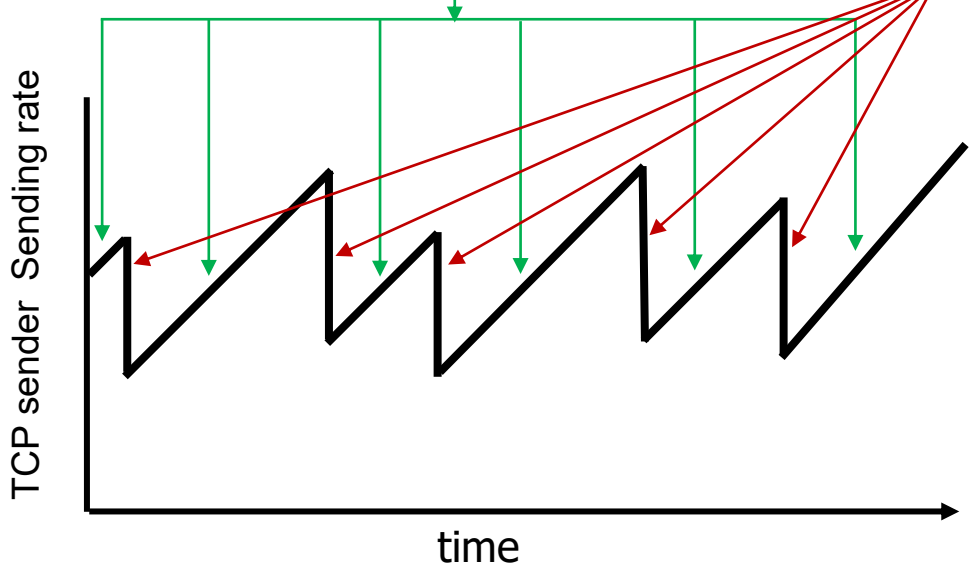
- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event.

Additive Increase

increase sending rate by 1 maximum segment size every RTT until loss detected

Multiplicative Decrease

cut sending rate in half at each loss event



- Chiu and Jain (1989): Let *w(t)* be the sending rate. *a (a > 0)* is the additive increase factor, and *b (0<b<1)* is the multiplicative decrease factor

$$w(t+1) = \begin{cases} w(t) + a & \text{if congestion is not detected} \\ w(t) \times b & \text{if congestion is detected} \end{cases}$$

TCP AIMD: more

Multiplicative decrease detail: sending rate is

- TCP Tahoe
  - Cut to 1 MSS (maximum segment size) when loss is detected by timeout.
  - Cut to 1MSS on loss detected by triple duplicate ACK

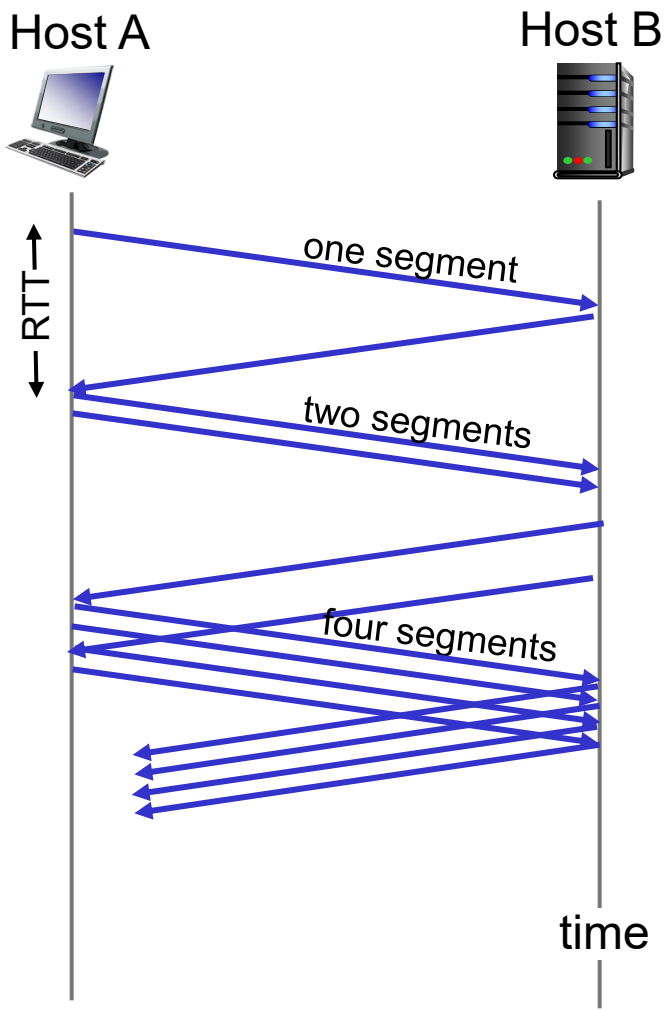
- TCP Reno
  - Cut in half on loss detected by triple duplicate ACK
  - Cut to 1 MSS (maximum segment size) when loss is detected by timeout.

TCP Congestion Control

- TCP maintains a **Congestion Window (CWnd) –** number of bytes the sender may have in the network at any time
- **Sender Window (SWnd) = Min (CWnd, RWnd)**
- RWnd – Receiver advertised window size

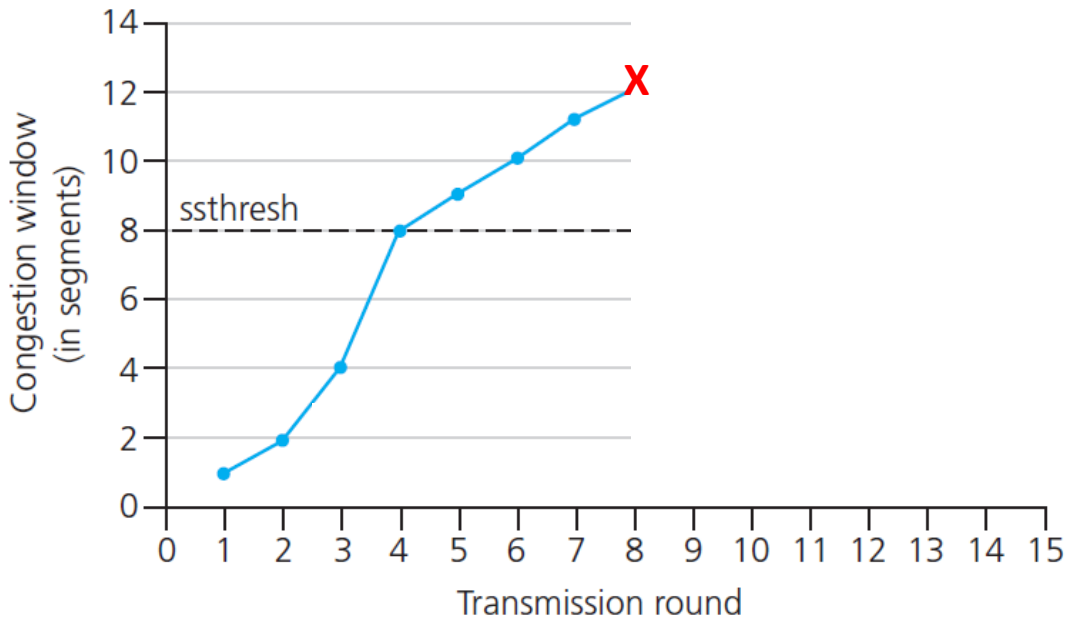
TCP slow start

- when connection begins, increase rate exponentially until first loss event:
  - initially **`cwnd`** = 1 MSS
  - double **`cwnd`** every RTT
  - done by incrementing **`cwnd`** for every ACK received
- *summary:* initial rate is slow, but ramps up exponentially fast



TCP: from slow start to congestion avoidance

The sender has two parameters for congestion control:
- **Congestion Window** (***cwnd***; Initial value is MSS bytes)
- **Threshhold Value** (***ssthresh***; Initial value is 65536 bytes)



Implementation:

- variable **`ssthresh`**
- on loss event, **`ssthresh`** is set to 1/2 of **`cwnd`** just before loss event

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Slow Start Cont..

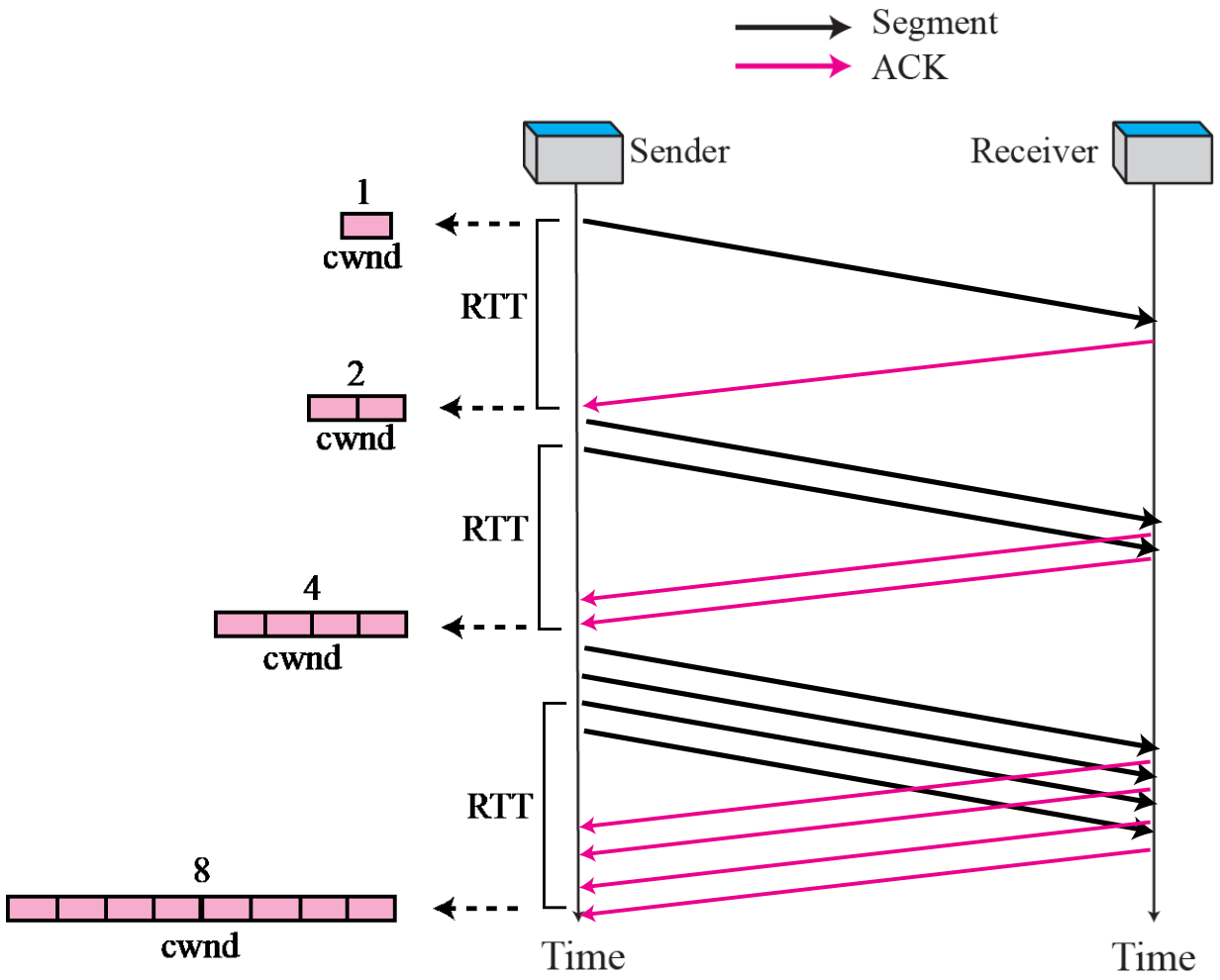
- Slow start causes exponential growth, eventually it will send too many packets into the network too quickly.
- To keep slow start under control, the sender keeps a threshold for the connection called the **slow start threshold (ssthresh).**
- Whenever a packet loss is detected by a timeout, the ssthresh is set to be half of the congestion window

Congestion Avoidance (Additive Increase)

- Whenever ssthresh is crossed, TCP switches from slow start to additive increase.
- Usually implemented with an partial increase for every segment that is acknowledged, rather than an increase of one segment per RTT.
- A common approximation is to increase Cwnd for additive increase as follows:

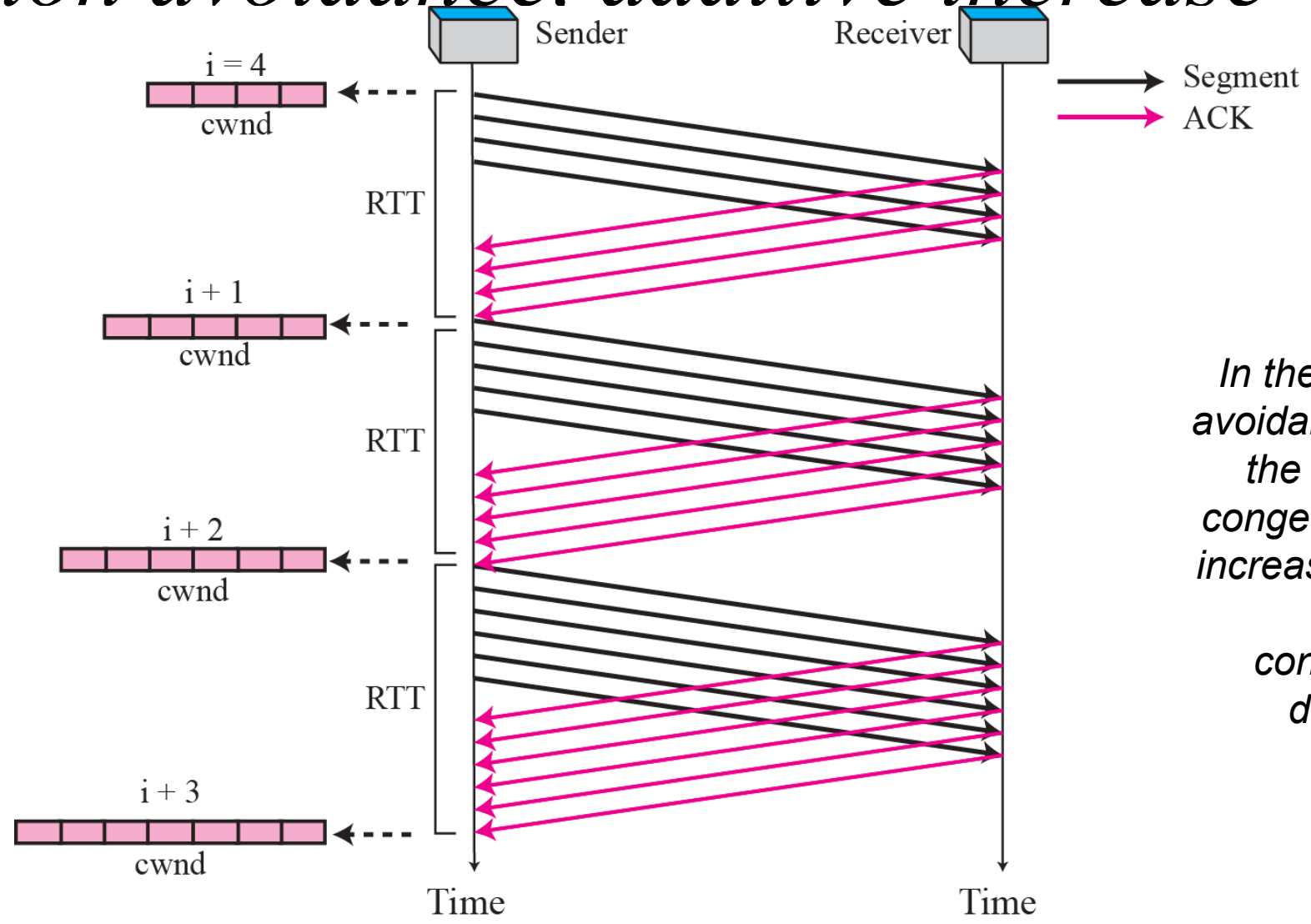
$$\boldsymbol{CWnd = Cwnd + \frac{MSS \times MSS}{CWnd}}$$

Congestion Control: Slow start, exponential increase



In the slow start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

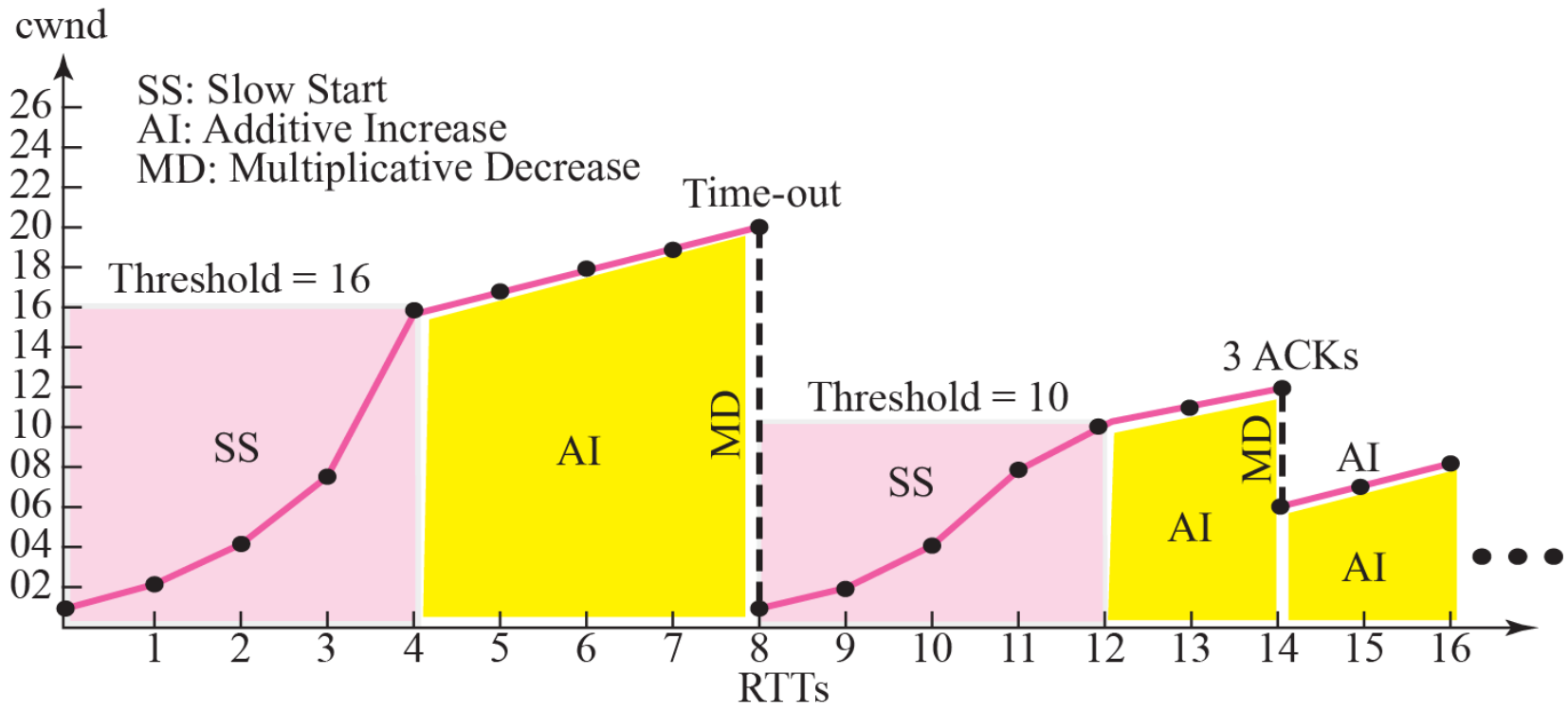
Congestion avoidance. additive increase



In the congestion avoidance algorithm the size of the congestion window increases additively until congestion is detected.

TCP/IP Protocol Suite

Congestion example



Fast Retransmission - TCP Tahoe

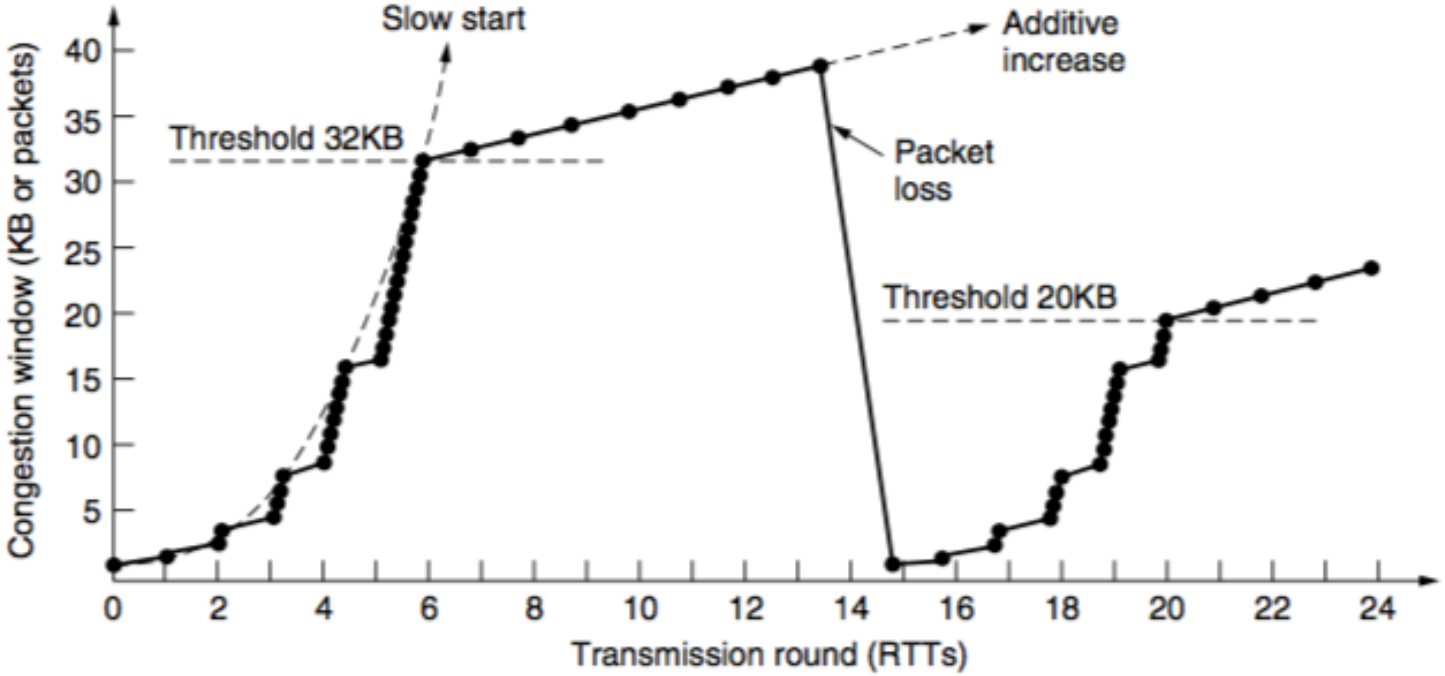
Use THREE DUPACK as the sign of congestion

Once 3 DUPACKs have been received,

Retransmit the lost packet (**fast retransmission)**

Set ssthresh as half of the current CWnd

Set CWnd to 1 MSS



Fast Recovery – TCP Reno

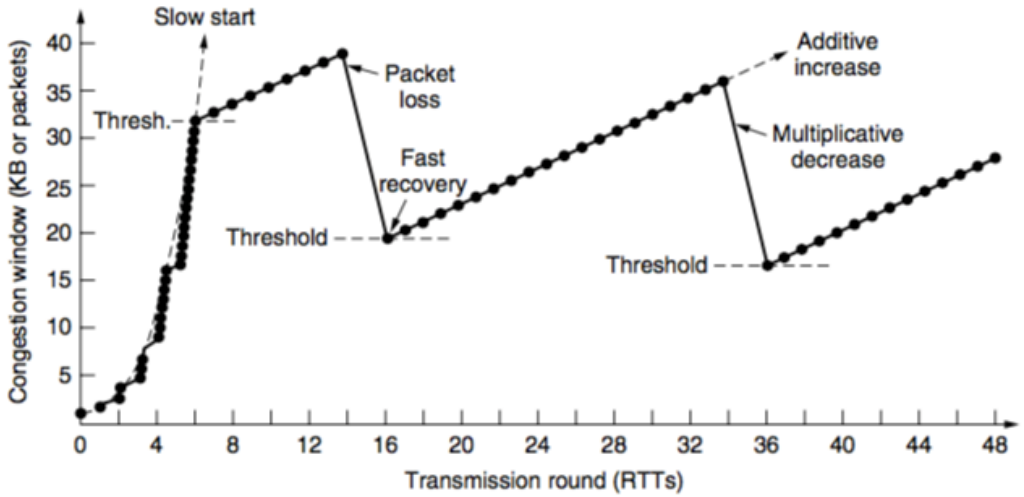
- **Once a congestion is detected through 3 DUPACKs, do TCP really need to set CWnd = 1 MSS ?**

- DUPACK means that **some segments are still flowing in the network** – a signal for temporary congestion, but not a prolonged one

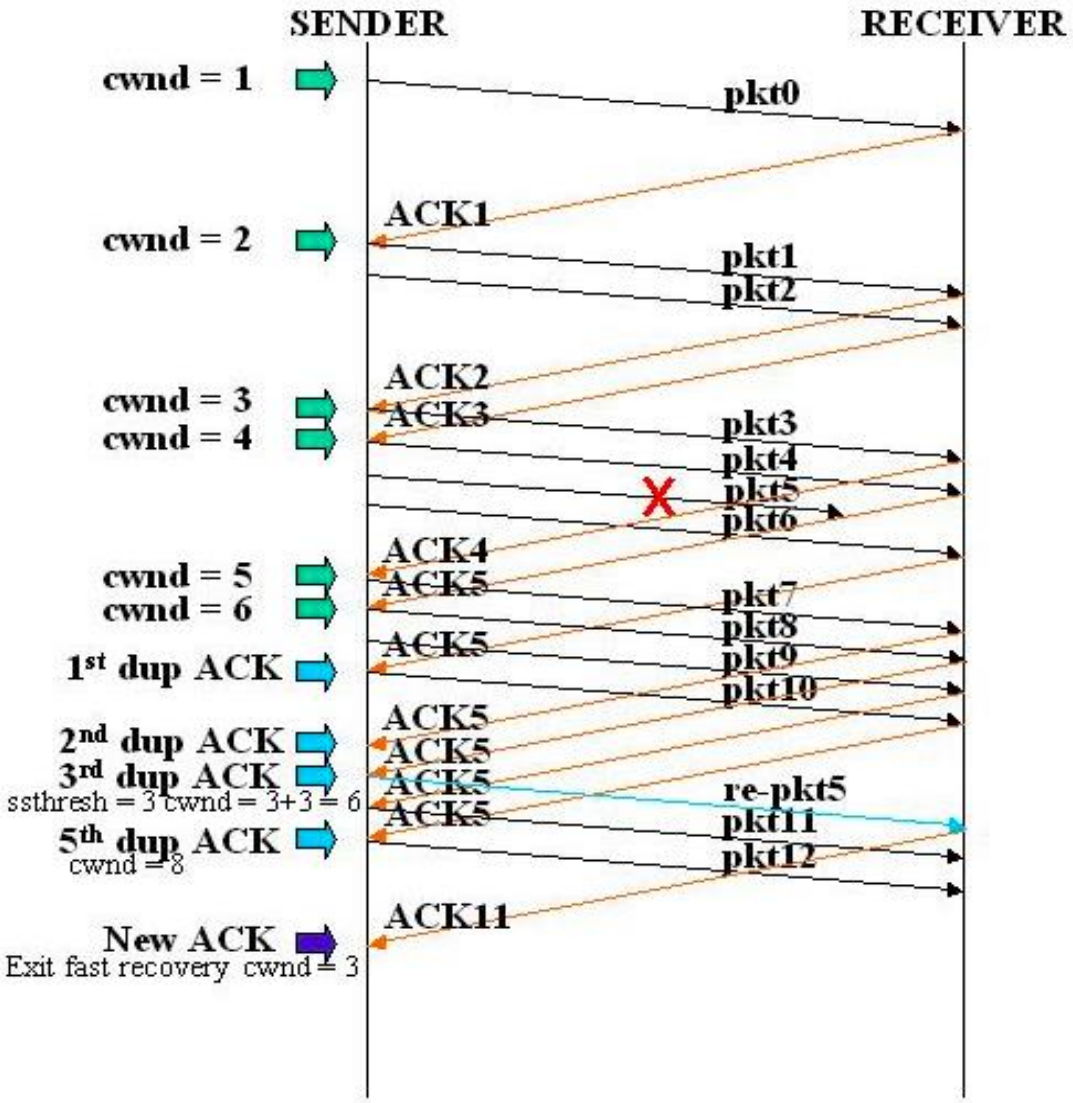
- Immediately transmit the lost segment (**fast retransmit)**, then transmit additional segments based on the DUPACKs received **(fast recovery)**

Fast Recovery – TCP Reno

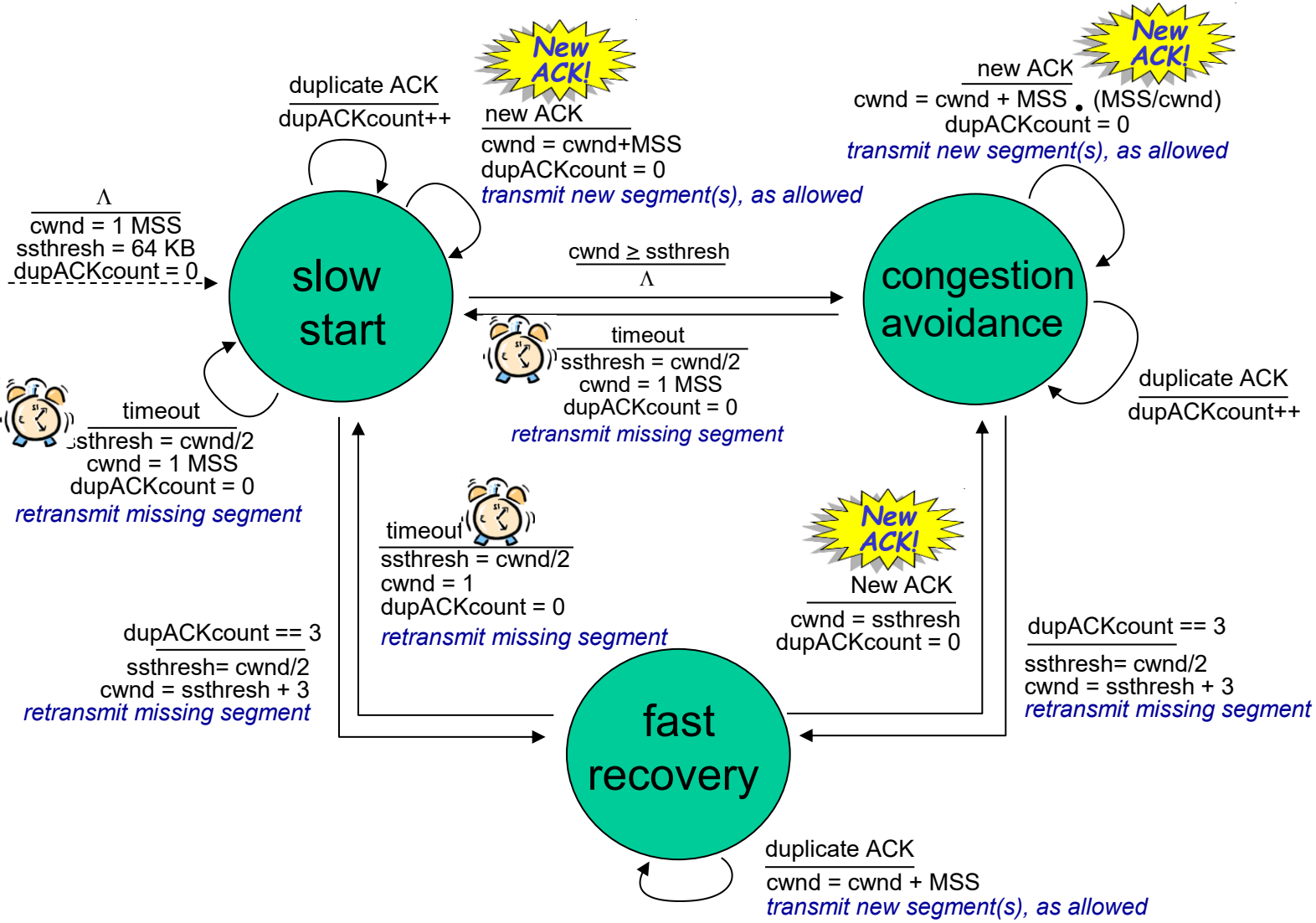
- **Fast recovery:**
- set ssthresh to half of the current congestion window. Retransmit the missing segment.
- set cwnd = ssthresh + 3.
- Each time another duplicate ACK arrives, set cwnd = cwnd + 1. Then, send a new data segment if allowed by the value of cwnd.
- Once receive a new ACK (an ACK which acknowledges all intermediate segments sent between the lost packet and the receipt of the first duplicate ACK), exit fast recovery. This causes setting cwnd to ssthresh (the ssthresh in step 1). Then, continue with linear increasing due to congestion avoidance algorithm.



Example: Fast Recovery – TCP Reno



Summary: TCP congestion control



TCP Congestion Control Algorithms

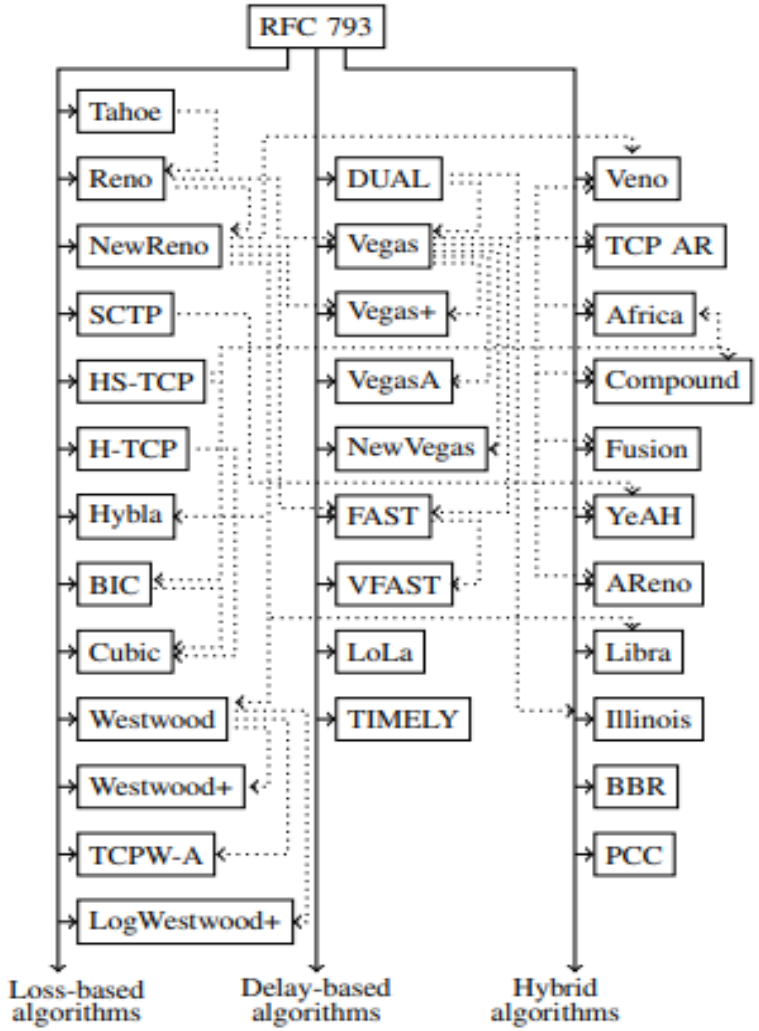
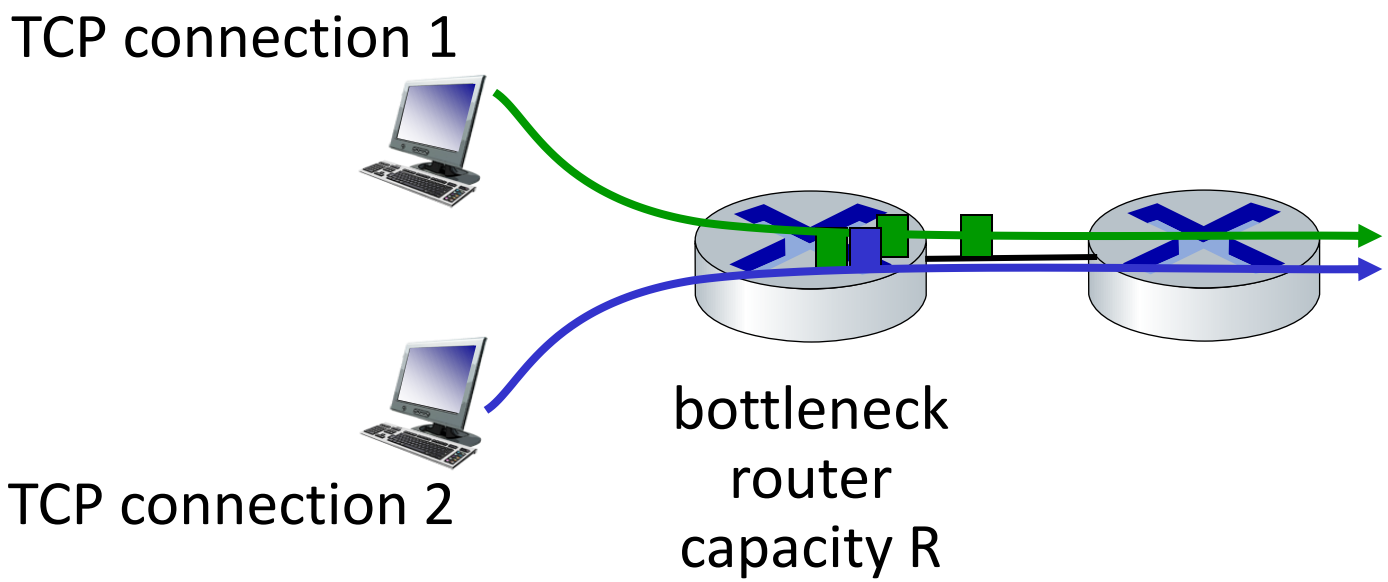


Fig. 3: Classification of different congestion control algorithms. Dotted arrows indicate that one was based on the other.

- TCP Cubic: Used by Many Linux systems

TCP fairness

Fairness goal: if *K* TCP sessions share same bottleneck link of bandwidth *R*, each should have average rate of *R/K*



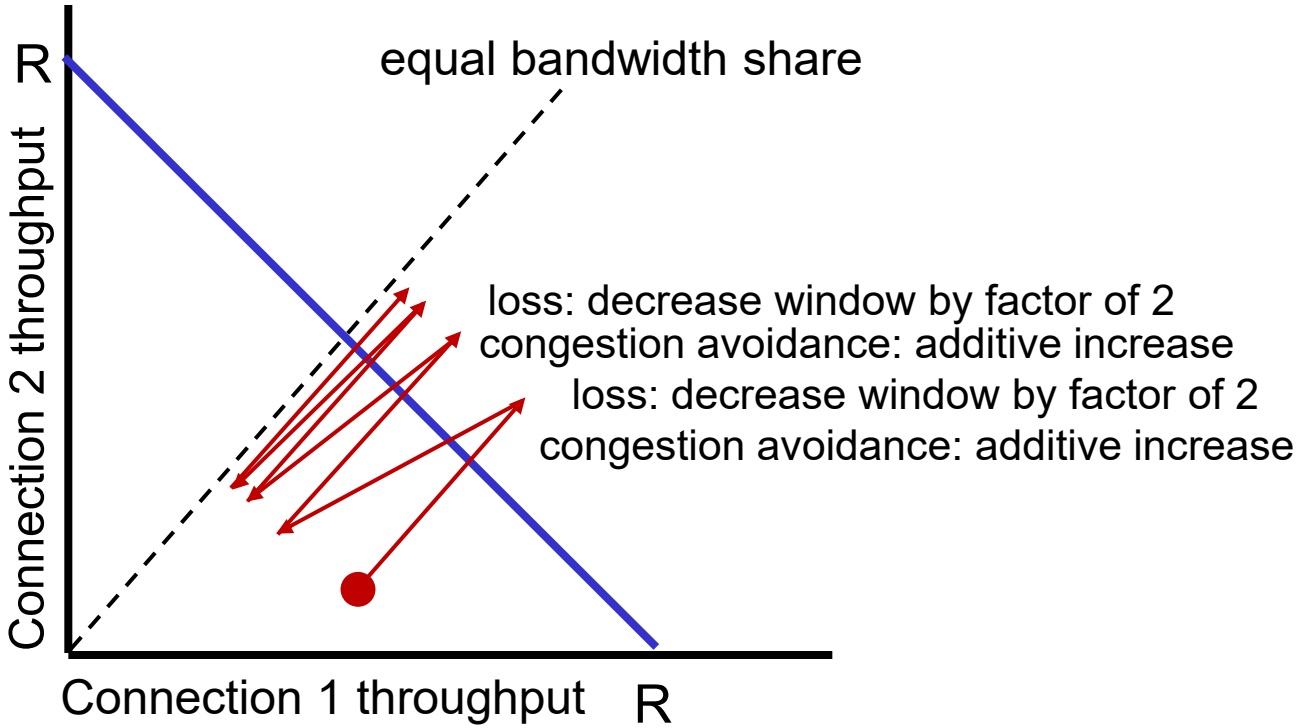
Goal:

1. Fairness: Every user should get a fair share of link
2. Capacity: Network link bandwidth should be well utilized

Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase – Increases Efficiency
- multiplicative decrease decreases throughput proportionally - Increases Fairness



Is TCP fair?

A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions

Fairness: must all network apps be “fair”?

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- there is no “Internet police” policing use of congestion control

Fairness, parallel TCP connections

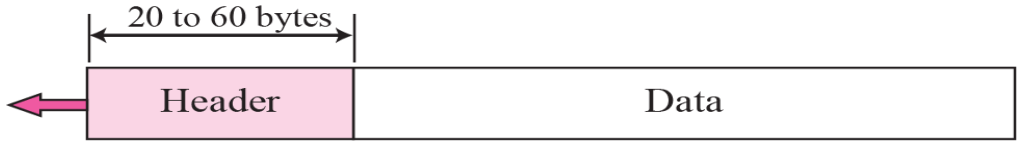
- application can open *multiple* parallel connections between two hosts
- web browsers do this , e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets R/2

TCP Options

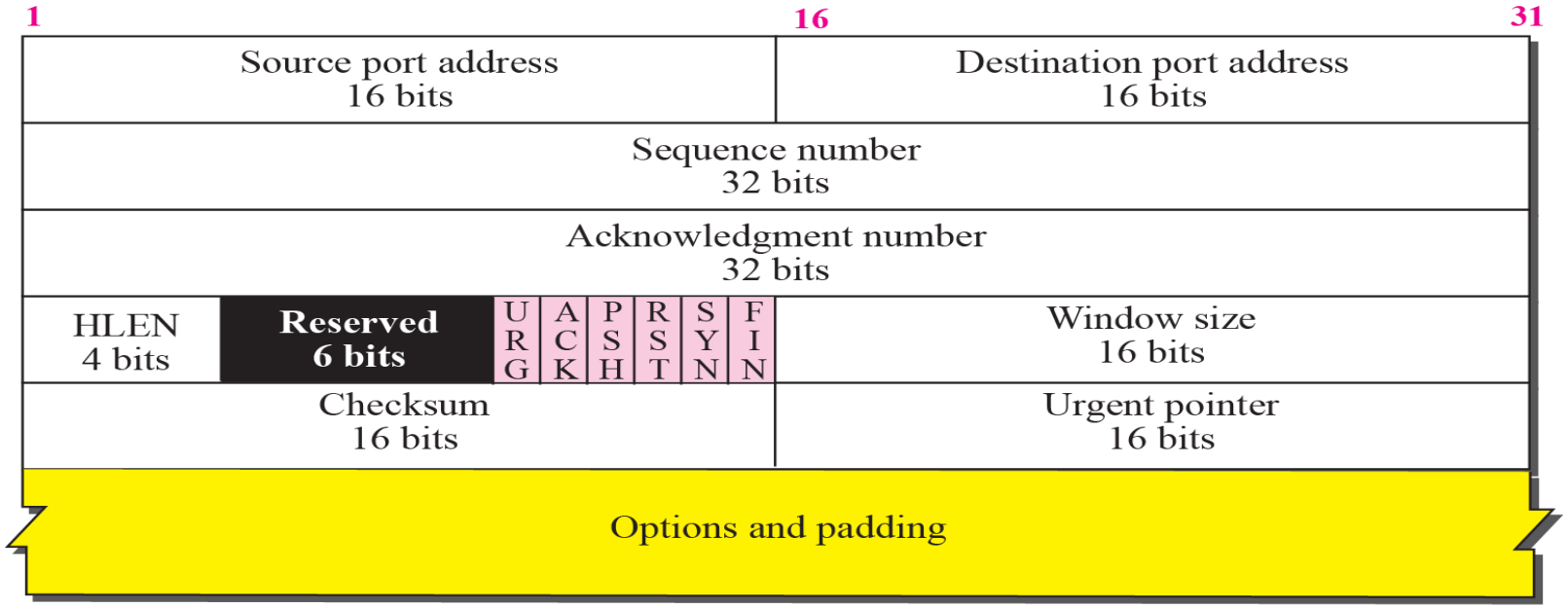
- The TCP header can have up to 40 bytes of optional information. Options convey additional information to the destination.
- We can define two categories of options: 1-byte options and multiple-byte options.
- The first category contains two types of options: end of option list and no operation.
- The second category, in most implementations, contains five types of options: maximum segment size, window scale factor, timestamp, SACK-permitted, and SACK.

TCP segment format

- Before discussing TCP in more detail, let us discuss the TCP packets themselves. A packet in TCP is called a segment.

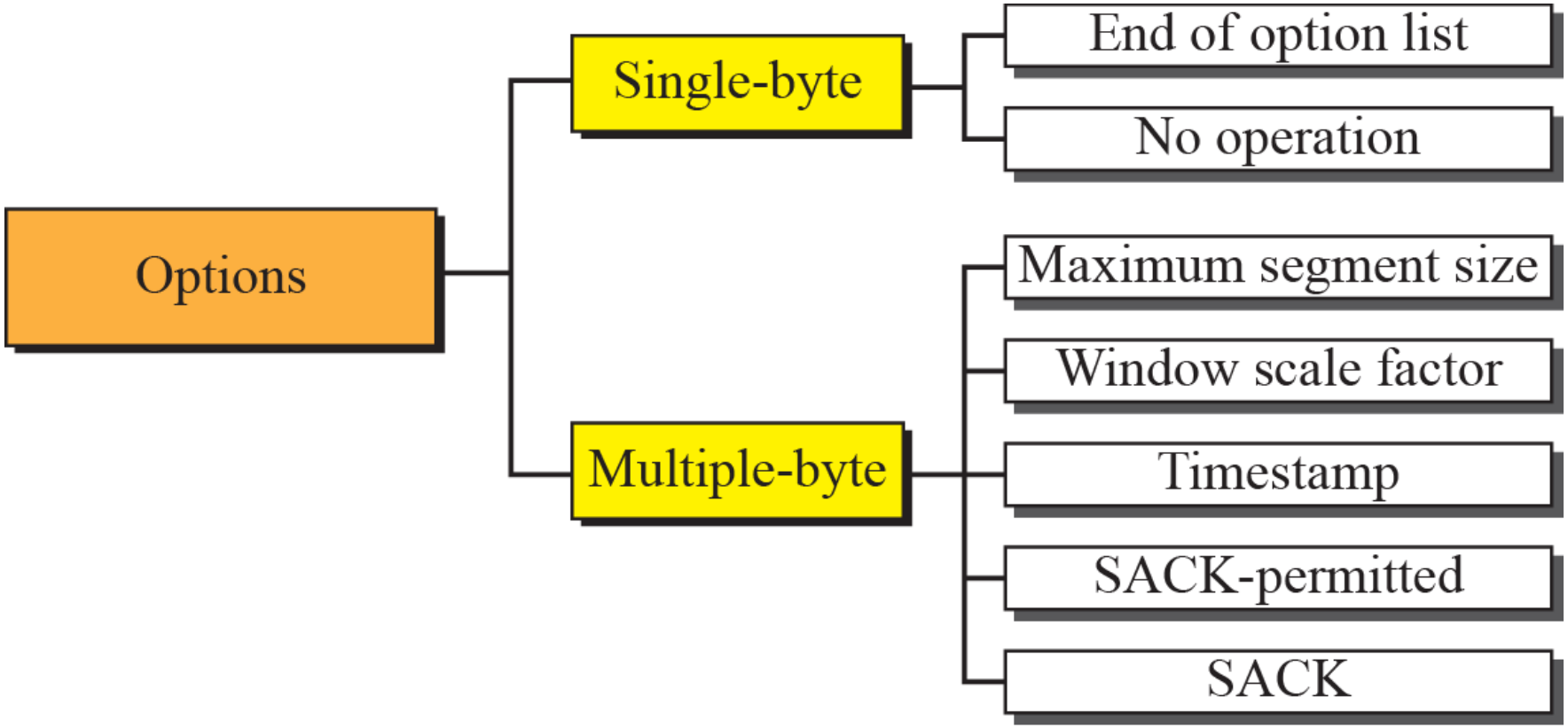


a. Segment



b. Header

Options

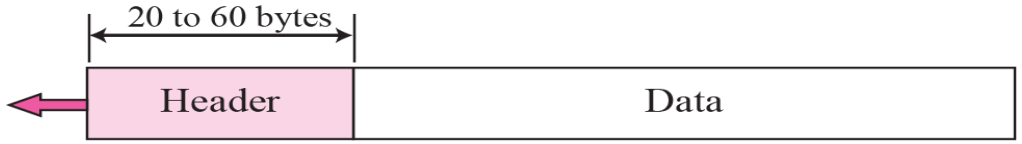

Options
Single-byte
End of option list
No operation
Multiple-byte
Maximum segment size
Window scale factor
Timestamp
SACK-permitted
SACK

TCP Options

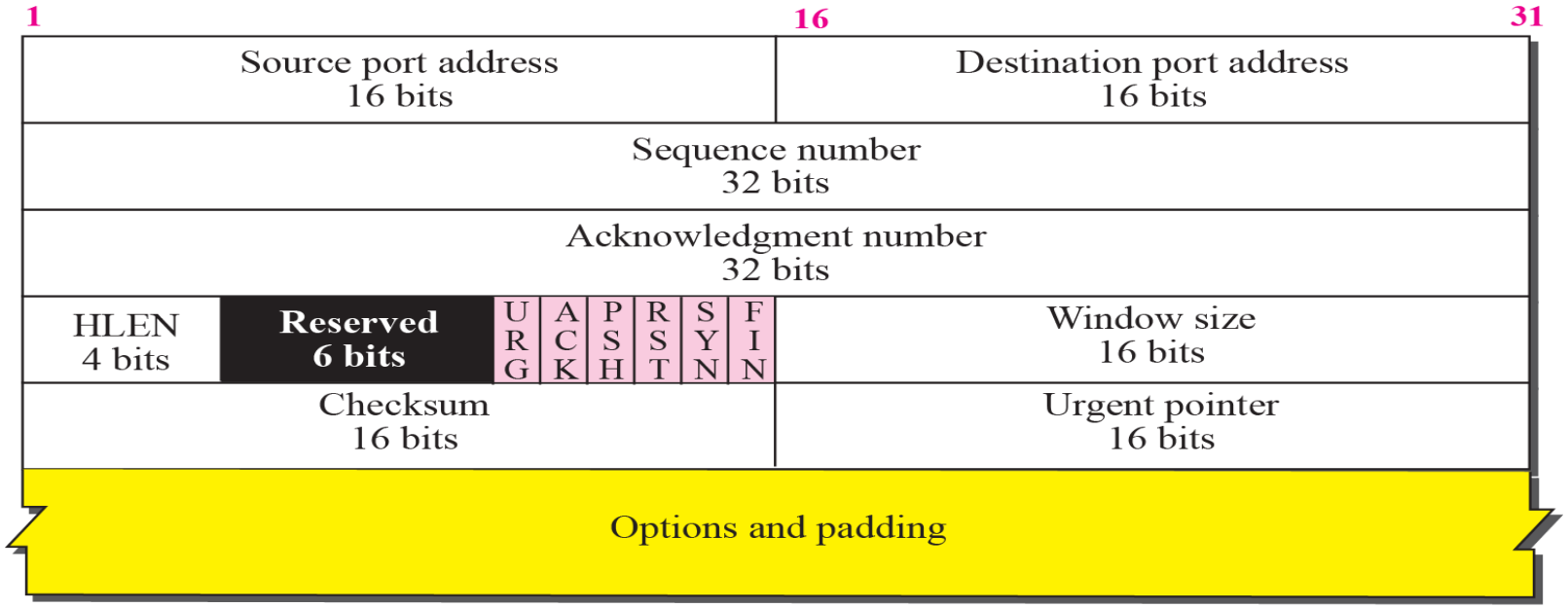
- The TCP header can have up to 40 bytes of optional information. Options convey additional information to the destination.
- We can define two categories of options: 1-byte options and multiple-byte options.
- The first category contains two types of options: end of option list and no operation.
- The second category, in most implementations, contains five types of options: maximum segment size, window scale factor, timestamp, SACK-permitted, and SACK.

TCP segment format

- Before discussing TCP in more detail, let us discuss the TCP packets themselves. A packet in TCP is called a segment.



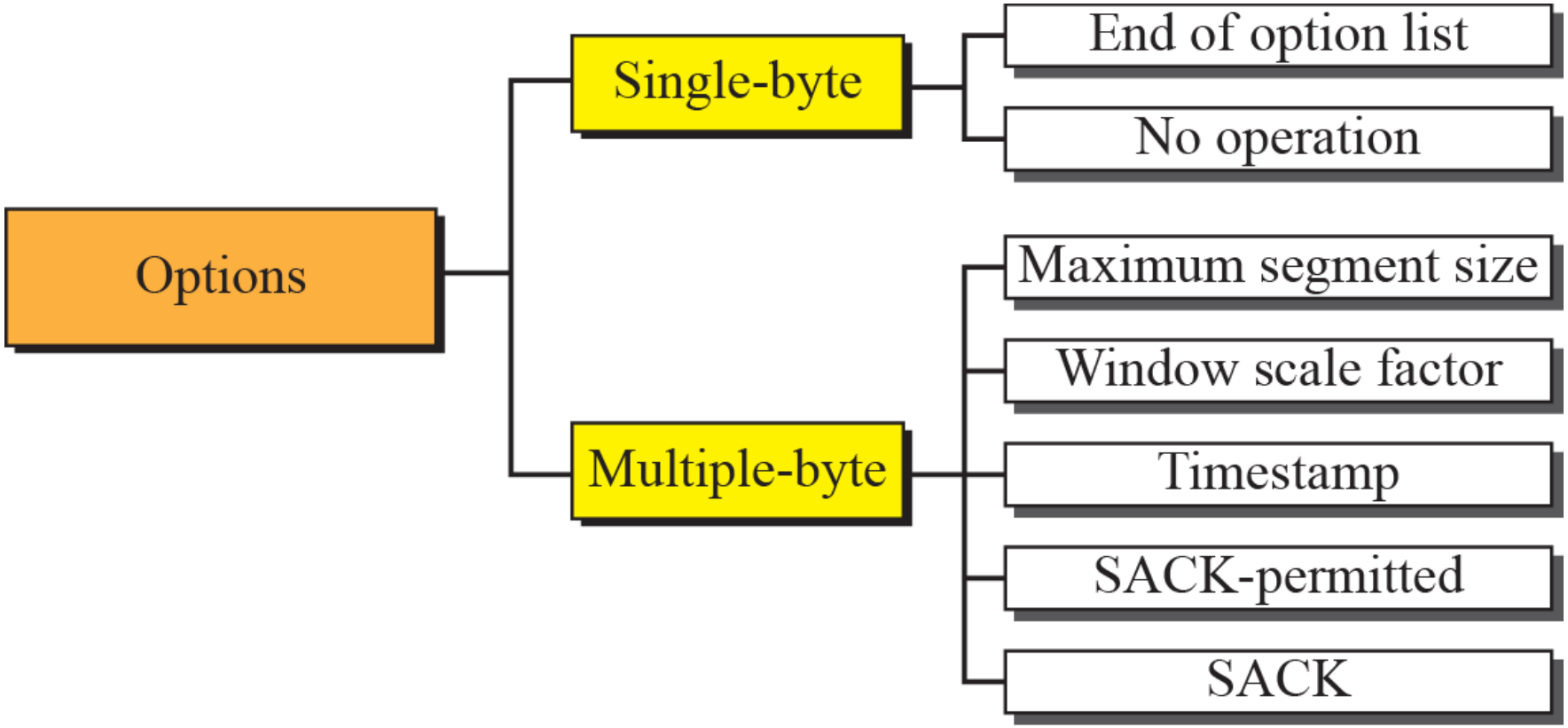
a. Segment



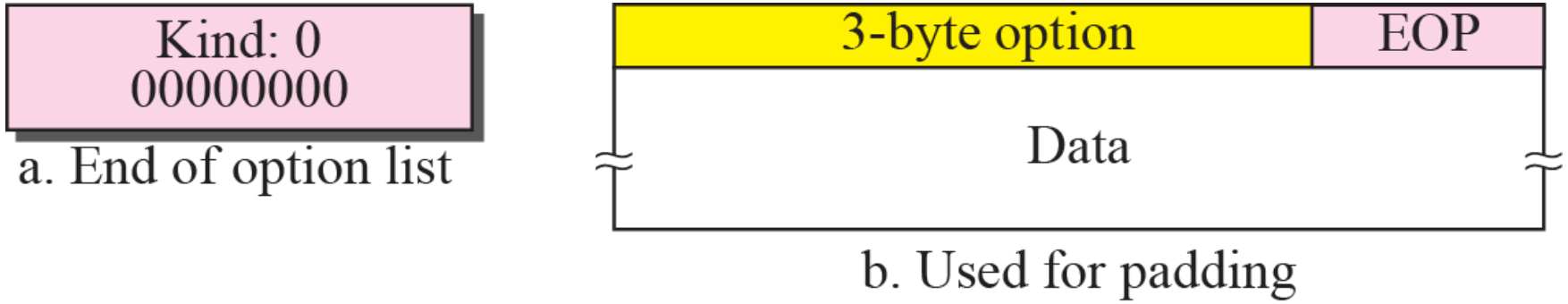
b. Header

TCP/IP Protocol Suite

Options


Options
Single-byte
End of option list
No operation
Multiple-byte
Maximum segment size
Window scale factor
Timestamp
SACK-permitted
SACK

End-of-option option

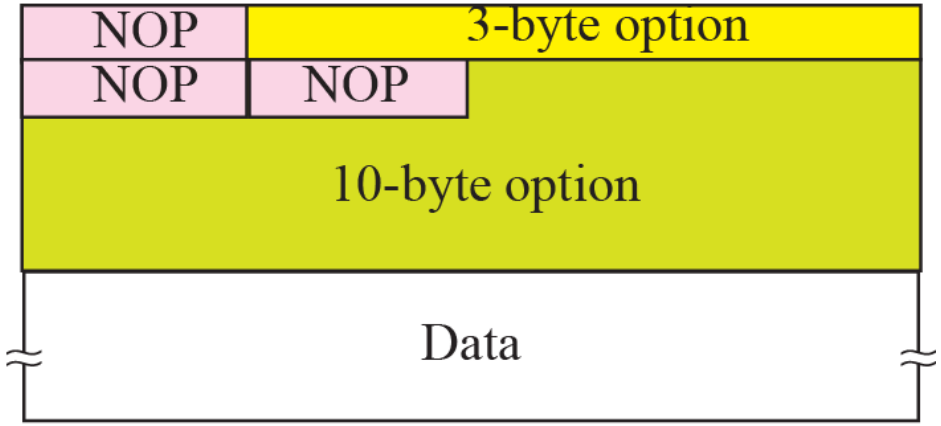


EOP can be used only once.

No-operation option

- It is used to separate the different options used within the TCP Option field.

a. No operation option



b. Used to align beginning of an option

NOP can be used more than once.

Minimum-segment-size option

1) The MSS (Maximum Segment Size) is defined as the largest block of data that a sender using TCP will send to the receiver.
2) When a connection is initiated a SYN segment is sent, in the process of sending a SYN segment, the sender has the option of announcing its MSS.
3) If a sender doesn't use the options field to declare the MSS then TCP assumes a default of 536bytes (minus the 20 byte TCP header).

Kind: 2 00000010	Length: 4 00000100	Maximum segment size
1 byte	1 byte	2 bytes

The value of MSS is determined during connection establishment and does not change during the connection.

Window-scale-factor option

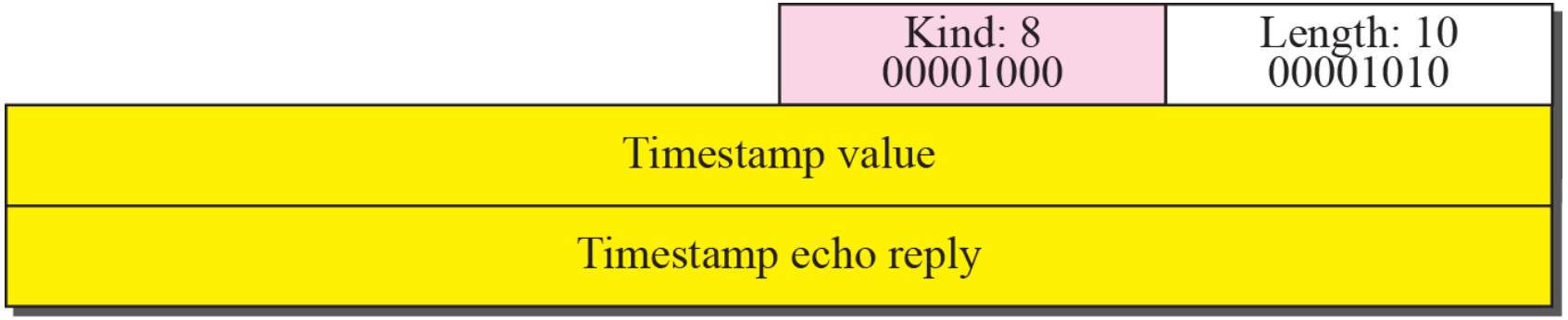
1. For lines with high bandwidth, high delay or high bandwidth-delay product, 64KB is not enough to keep the sender busy all the time.
2. In RFC 1323, the use of the options field is proposed permitting the use of a window scale factor.
3. This scale factor permits shifting the window size up to 14 bits to the left therefore permitting window sizes of up to $2^{(16+14)} = 2^{30}$ bytes.

Example window size =65535, and scale factor = 2 (2^ 2 = 4 multiply by 4 to window size to calculate the actual window size)

Kind: 3 00000011	Length: 3 00000011	Scale factor
1 byte	1 byte	1 byte

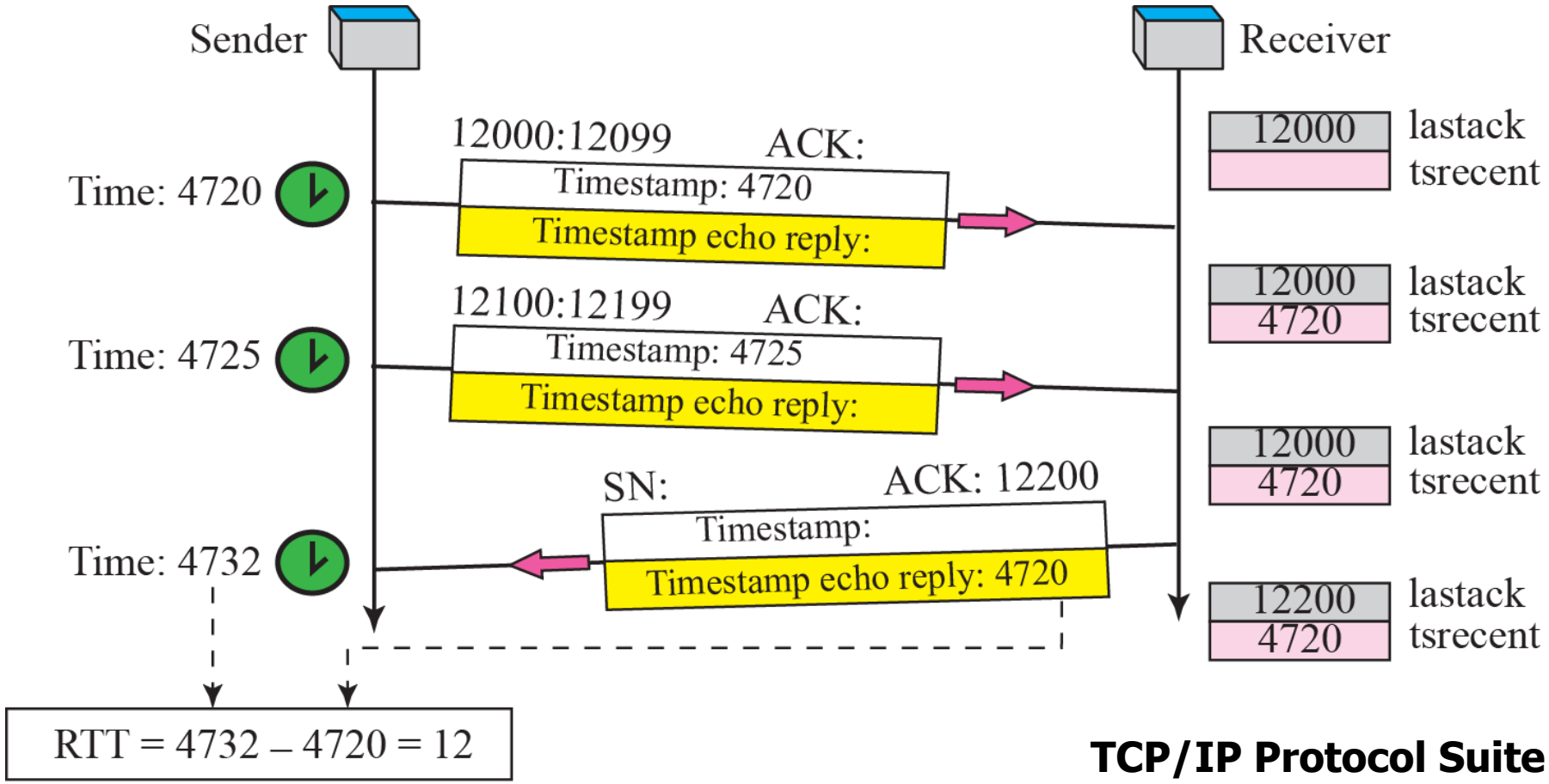
The value of the window scale factor can be determined only during connection establishment; it does not change during the connection.

Timestamp option

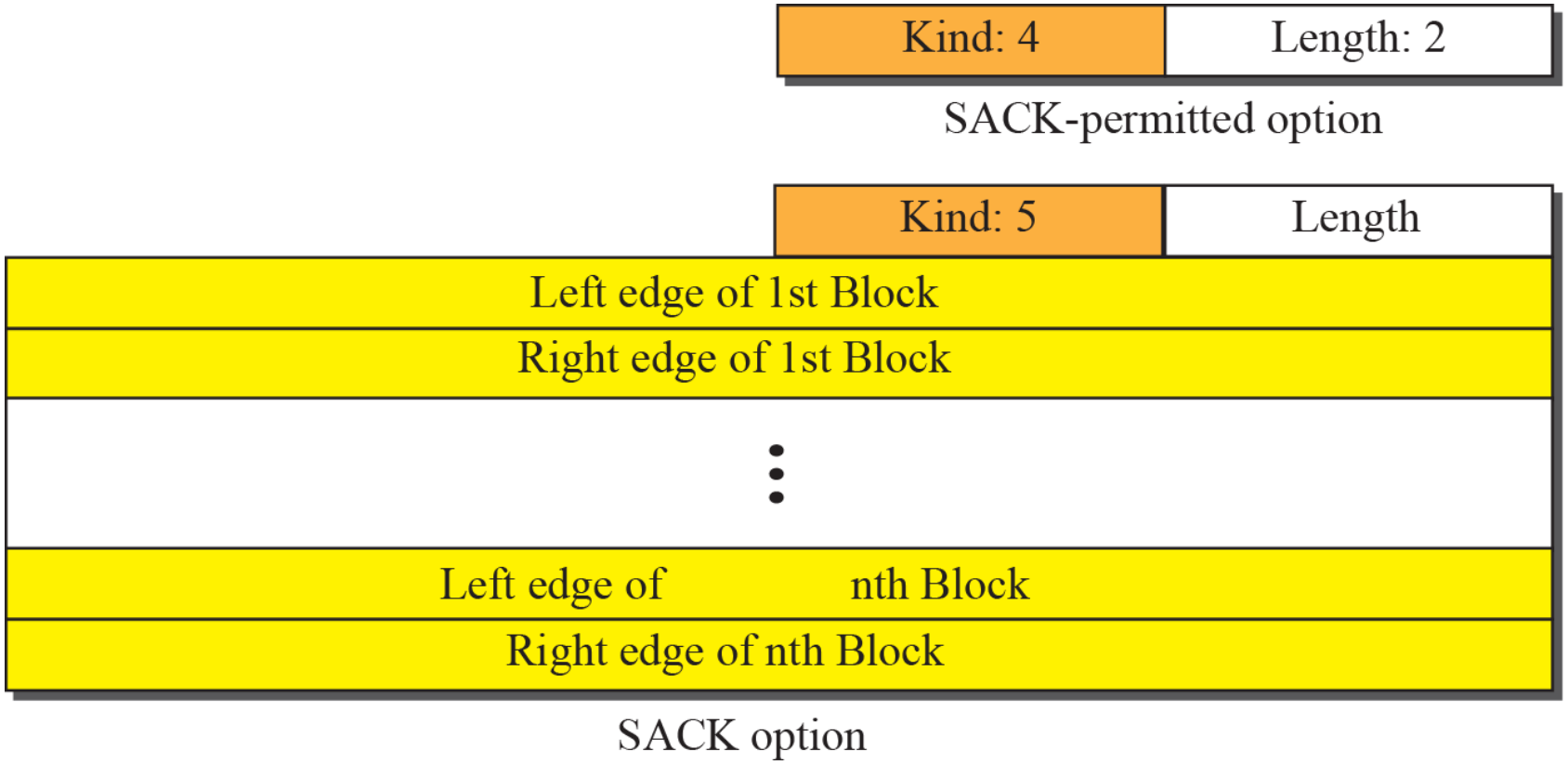


One application of the timestamp option is the calculation of round-trip time (RTT).

Figure shows an example that calculates the round-trip time for one end. Everything must be flipped if we want to calculate the RTT for the other end.



Selective ACK (*SACK*)



Let us see how the SACK option is used to list out-of-order blocks. In Figure an end has received five segments of data.

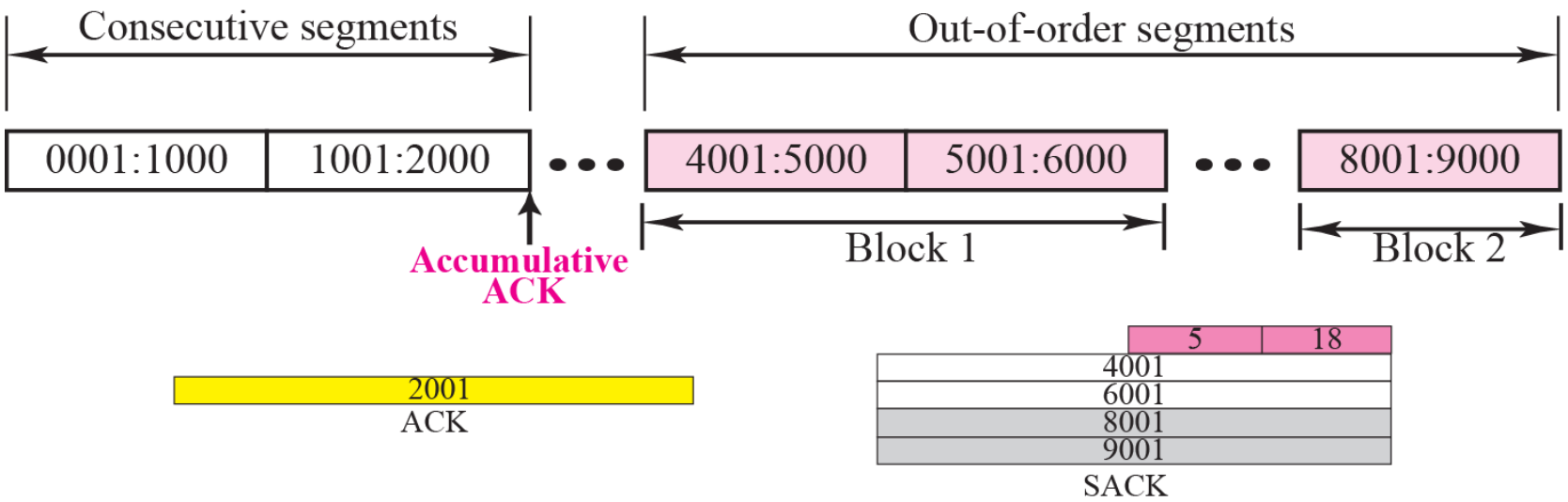


Figure shows how a duplicate segment can be detected with a combination of ACK and SACK.

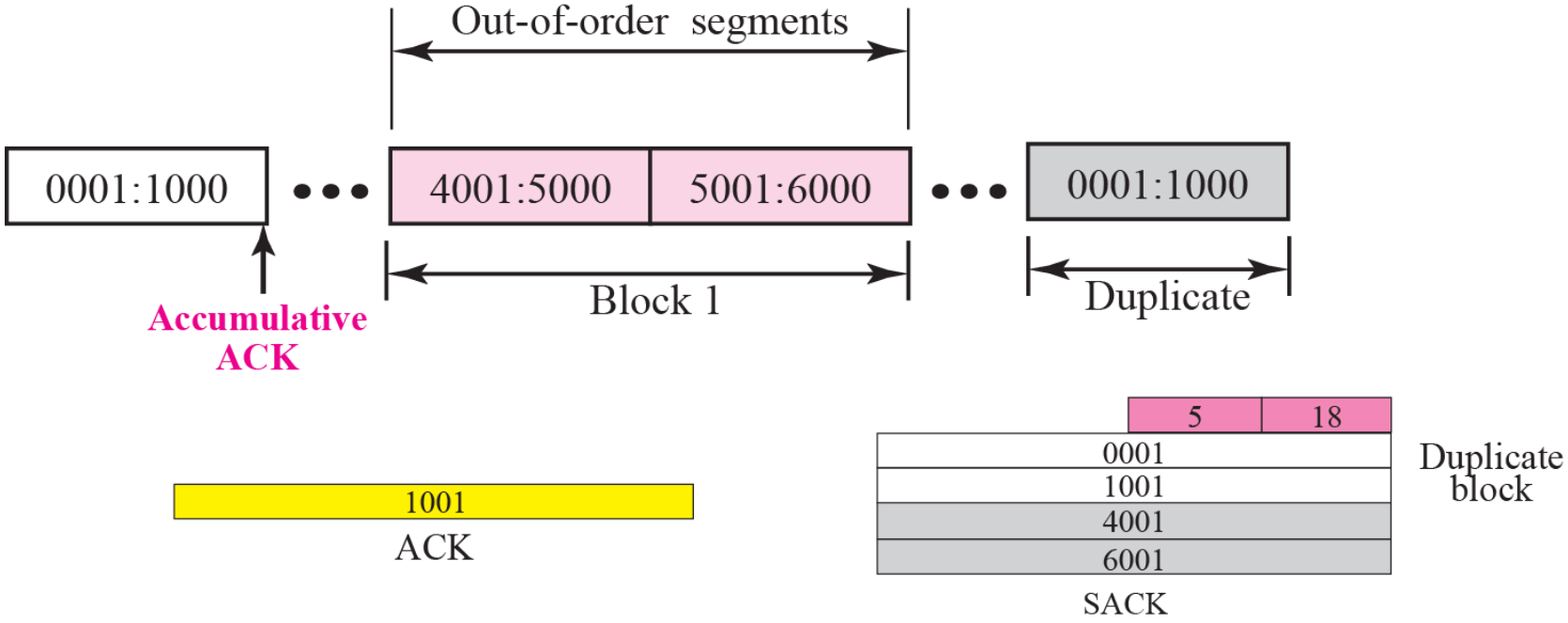
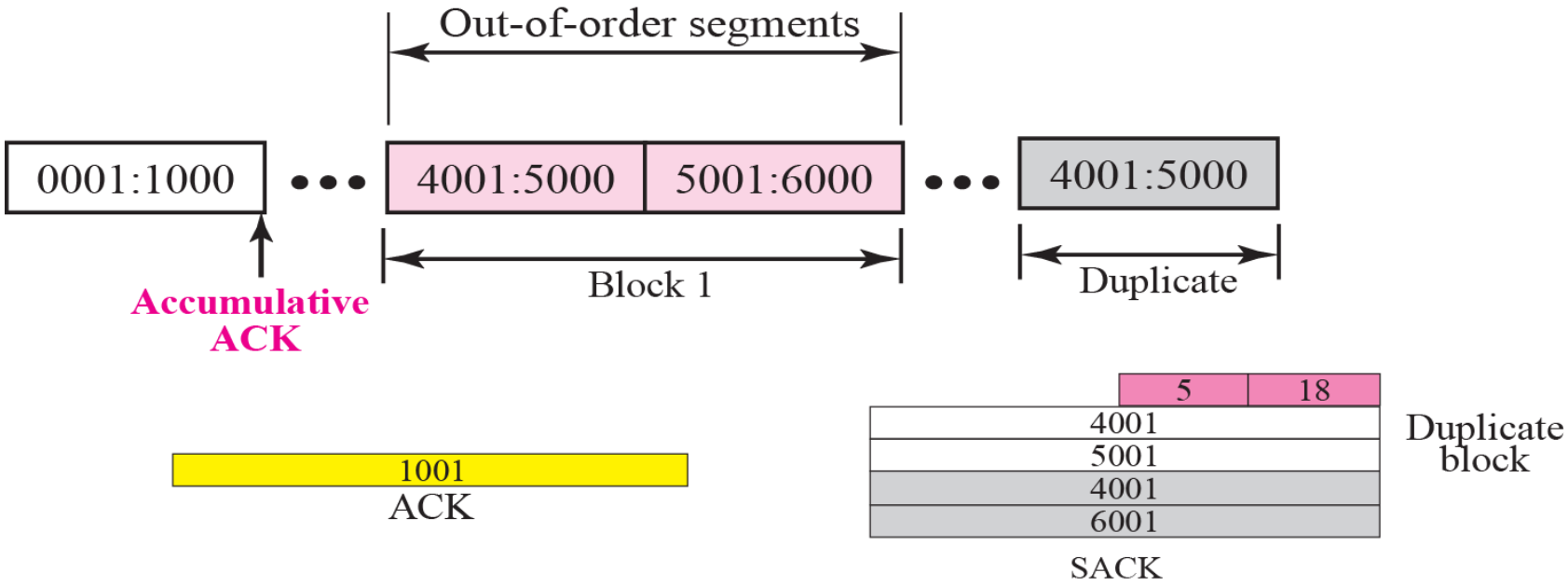


Figure shows what happens if one of the segments in the out-of-order section is also duplicated. In this example, one of the segments (4001:5000) is duplicated.

The SACK option announces this duplicate data first and then the out-of-order block.